Data Science II Assignment 1

Ye Wen

Setup

```r
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.2     v tibble    3.3.0
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.1.0
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
train <- read_csv("./data/housing_training.csv")
```

```
## Rows: 1440 Columns: 26
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (4): Overall_Qual, Kitchen_Qual, Fireplace_Qu, Exter_Qual
## dbl (22): Gr_Liv_Area, First_Flr_SF, Second_Flr_SF, Total_Bsmt_SF, Low_Qual_...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
test <- read_csv("./data/housing_test.csv")
```

```
## Rows: 959 Columns: 26
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (4): Overall_Qual, Kitchen_Qual, Fireplace_Qu, Exter_Qual
## dbl (22): Gr_Liv_Area, First_Flr_SF, Second_Flr_SF, Total_Bsmt_SF, Low_Qual_...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
# Ordinal encoding based on dictionary.txt levels
train <- train %>%
  mutate(
    Overall_Qual = recode(Overall_Qual,
                          "Fair"=1, "Below_Average"=2, "Average"=3,
                          "Above_Average"=4, "Good"=5, "Very_Good"=6,
                          "Excellent"=7, "Very_Excellent"=8),
    Kitchen_Qual = recode(Kitchen_Qual,
                          "Fair"=1, "Typical"=2, "Good"=3, "Excellent"=4),
```

```r
    Fireplace_Qu = recode(Fireplace_Qu,
                          "No_Fireplace"=0, "Poor"=1, "Fair"=2, "Typical"=3,
                          "Good"=4, "Excellent"=5),
    Exter_Qual = recode(Exter_Qual,
                        "Poor"=1, "Fair"=2, "Typical"=3, "Good"=4, "Excellent"=5)
  )

test <- test %>%
  mutate(
    Overall_Qual = recode(Overall_Qual,
                          "Fair"=1, "Below_Average"=2, "Average"=3,
                          "Above_Average"=4, "Good"=5, "Very_Good"=6,
                          "Excellent"=7, "Very_Excellent"=8),
    Kitchen_Qual = recode(Kitchen_Qual,
                          "Fair"=1, "Typical"=2, "Good"=3, "Excellent"=4),
    Fireplace_Qu = recode(Fireplace_Qu,
                          "No_Fireplace"=0, "Poor"=1, "Fair"=2, "Typical"=3,
                          "Good"=4, "Excellent"=5),
    Exter_Qual = recode(Exter_Qual,
                        "Poor"=1, "Fair"=2, "Typical"=3, "Good"=4, "Excellent"=5)

  )

y_train <- train$Sale_Price
y_test <- test$Sale_Price

x_train <- train %>% select(-Sale_Price)
x_test <- test %>% select(-Sale_Price)
```

Question (a)

Fit a lasso model on the training data. Report the selected tuning parameter and the test error. When the 1SE rule is applied, how many predictors are included in the model?

Answer

The selected tuning parameter was $\lambda_{min} = 136.56$. The model has a test MSE of 5.2967×10^8 and a RMSE of 23014.59. Applying the 1SE rule ($\lambda_{1se} = 2680.78$) yields a model with 14 nonzero predictors.

Code

```r
library(glmnet)
```

```
## Loading required package: Matrix

##
## Attaching package: 'Matrix'

## The following objects are masked from 'package:tidyr':
##
##     expand, pack, unpack

## Loaded glmnet 4.1-10
```

```r
set.seed(1)
Xtr <- model.matrix(Sale_Price ~ ., data=train)[, -1]
```

```r
Xte <- model.matrix(Sale_Price ~ ., data=test)[, -1]

cv_lasso <- cv.glmnet(
  x = Xtr, y = y_train,
  alpha = 1,
  nfolds = 10,
  standardize = TRUE
)

lambda_min <- cv_lasso$lambda.min
lambda_1se <- cv_lasso$lambda.1se

lambda_min
```

```
## [1] 136.5625
```

```r
lambda_1se
```

```
## [1] 2680.777
```

```r
# test error at lambda.min
pred_min <- predict(cv_lasso, newx = Xte, s = "lambda.min")
mse_min  <- mean((y_test - pred_min)^2)
rmse_min <- sqrt(mse_min)

mse_min
```

```
## [1] 529671127
```

```r
rmse_min
```

```
## [1] 23014.59
```

```r
# how many predictors in the model with 1SE rule
beta_1se <- coef(cv_lasso, s = "lambda.1se")
num_pred_1se <- sum(beta_1se[-1, 1] != 0)
num_pred_1se
```

```
## [1] 14
```

Question (b)

Fit an elastic net model on the training data. Report the selected tuning parameters and the test error. Is it possible to apply the 1SE rule to select the tuning parameters for elastic net? If the 1SE rule is applicable, implement it to select the tuning parameters. If not, explain why.

Answer

The elastic net model selected $\alpha = 0.5$ with $\lambda_{min} = 171.53$ and $\lambda_{1se} = 4055.82$. The resulting test error was $MSE = 5.3122 \times 10^8$ and $RMSE = 23048.12$. The 1SE rule is only applicable to λ conditional on a fixed α. Therefore, after selecting $\alpha = 0.5$, we can apply the 1SE rule.

Code

```r
alphas <- c(0.1, 0.3, 0.5, 0.7, 0.9, 0.95, 0.99)

set.seed(1)
```

```r
cv_list <- lapply(alphas, function(a) {
  cv.glmnet(Xtr, y_train, alpha = a, nfolds = 10, standardize = TRUE)
})

cv_mins <- sapply(cv_list, function(obj) min(obj$cvm))
best_i <- which.min(cv_mins)

best_alpha <- alphas[best_i]
best_cv <- cv_list[[best_i]]

best_alpha
```

```
## [1] 0.5
```

```r
best_cv$lambda.min
```

```
## [1] 171.5305
```

```r
best_cv$lambda.1se
```

```
## [1] 4055.82
```

```r
pred_en <- predict(best_cv, newx = Xte, s = "lambda.min")
mse_en  <- mean((y_test - pred_en)^2)
rmse_en <- sqrt(mse_en)

mse_en
```

```
## [1] 531215703
```

```r
rmse_en
```

```
## [1] 23048.12
```

Question (c)

Fit a partial least squares model on the training data and report the test error. How many components are included in your model?

Answer

The partial least squares model included 8 components, and had test error $MSE \approx 5.332 \times 10^8$ and $RMSE = 23091.11$.

Code

```r
library(pls)
```

```
## Warning: package 'pls' was built under R version 4.5.2
```

```
##
## Attaching package: 'pls'
```

```
## The following object is masked from 'package:stats':
##
##     loadings
```

```r
set.seed(1)
pls_fit <- plsr(
```

```r
  Sale_Price ~ .,
  data = train,
  scale = TRUE,
  validation = "CV"
)

rmsep_obj <- RMSEP(pls_fit)
rmsep_vals <- rmsep_obj$val[1,1,]
best_comp <- which.min(rmsep_vals) - 1
best_comp
```

```
## 8 comps
##       8
```

```r
pred_pls <- predict(pls_fit, newdata = test, ncomp = best_comp)
mse_pls  <- mean((y_test - pred_pls)^2)
rmse_pls <- sqrt(mse_pls)

mse_pls
```

```
## [1] 533199330
```

```r
rmse_pls
```

```
## [1] 23091.11
```

Question (d)

Choose the best model for predicting the response and explain your choice.

Answer

Based on the test errors in (a)-(c), **the lasso model performs best**. Additionally, lasso is also more interpretable because it performs variable selection.

Question (e)

If R package "caret" was used for the lasso in (a), retrain this model using R package "glmnet" and vice versa. Compare the selected tuning parameters between the two software approaches. Should there be discrepancies in the chosen parameters, discuss potential reasons for these differences.

Answer

Using `glmnet`, the lasso selected $\lambda_{min} = 136.56$ and $\lambda_{1se} = 2680.78$. Using `caret`, the selected tuning parameter was $\lambda = 124.59$. These lambda values are close but not identical, which makes sense, because: 1. They may try different candidate λ values. 2. They may split the CV folds differently. 3. They may standardize predictors differently. 4. They may handle categorical variables and dummy coding differently.

Nonetheless, their RMSE is almost the same, which is what matters for prediction.

Code

```r
library(caret)
```

```
## Loading required package: lattice
```

```
##
## Attaching package: 'caret'

## The following object is masked from 'package:pls':
##
##     R2

## The following object is masked from 'package:purrr':
##
##     lift
```

```r
set.seed(1)
ctrl <- trainControl(method = "cv", number = 10)

lasso_caret <- train(
  x = x_train, y = y_train,
  method = "glmnet",
  trControl = ctrl,
  preProcess = c("center","scale"),
  tuneGrid = expand.grid(alpha = 1, lambda = exp(seq(log(1e-3), log(1e3), length.out = 200)))
)
```

```
## Warning: Setting row names on a tibble is deprecated.
## Setting row names on a tibble is deprecated.
## Setting row names on a tibble is deprecated.
## Setting row names on a tibble is deprecated.
## Setting row names on a tibble is deprecated.
## Setting row names on a tibble is deprecated.
## Setting row names on a tibble is deprecated.
## Setting row names on a tibble is deprecated.
## Setting row names on a tibble is deprecated.
## Setting row names on a tibble is deprecated.
## Setting row names on a tibble is deprecated.
```

```r
lasso_caret$bestTune  # selected (alpha=1, lambda=...)
```

```
##     alpha   lambda
## 170     1 124.5883
```

```r
pred_test <- predict(lasso_caret, newdata = x_test)
mse_test  <- mean((y_test - pred_test)^2)
rmse_test <- sqrt(mse_test)

mse_test
```

```
## [1] 530083311
```

```r
rmse_test
```

```
## [1] 23023.54
```